Effortless Distributed Computing in Python

IAstro – Porto – July 2026

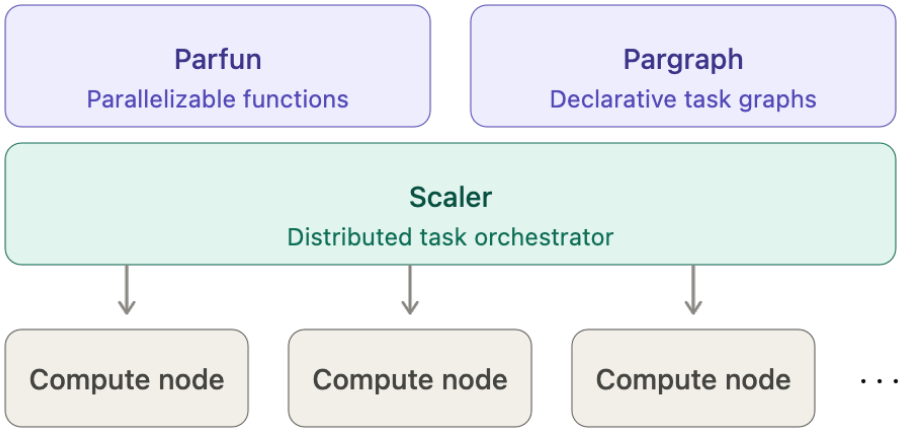
Raphael J.
https://raphaelj.be

👋 Nice to meet you

- **Raphael from Belgium** 🇧🇪
- Freelance **computer scientist**
- Past 2y+ working on **new parallelization libraries** for Python
  - Incubated by the **Linux Foundation (FINOS)**
  - **Designed for scientists / mathematicians**, not computer scientists
  - 🚧 Still under development (beta!)

Menu

- **Parallel and distributed computing**
  What it is, why and how
- **Parfun**
  Parallel Python functions
- **Pargraph**
  Distributed computational graphs
- **Scaler**
  Leveraging multiple computers
- **Astrophysics experiment**



Parallel and distributed computing

Parallel and distributed computing

Computer science is **sequential by default**

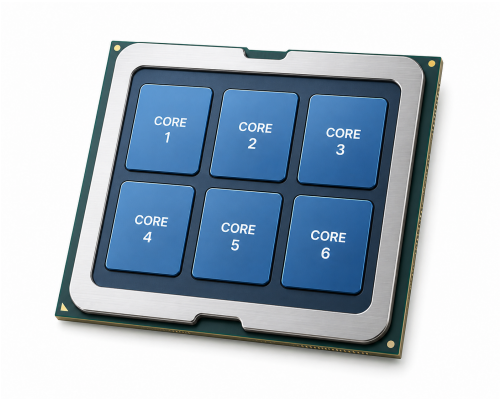
```
def is_prime(n: int) -> bool:
    if n < 2:
        return False

    for i in range(2, int(sqrt(n)) + 1):
        if n % i == 0:
            return False

    return True
```

"Do this then do that"

Parallel and distributed computing

Modern computers are **massively parallel**

Computer
4 to 64 cores

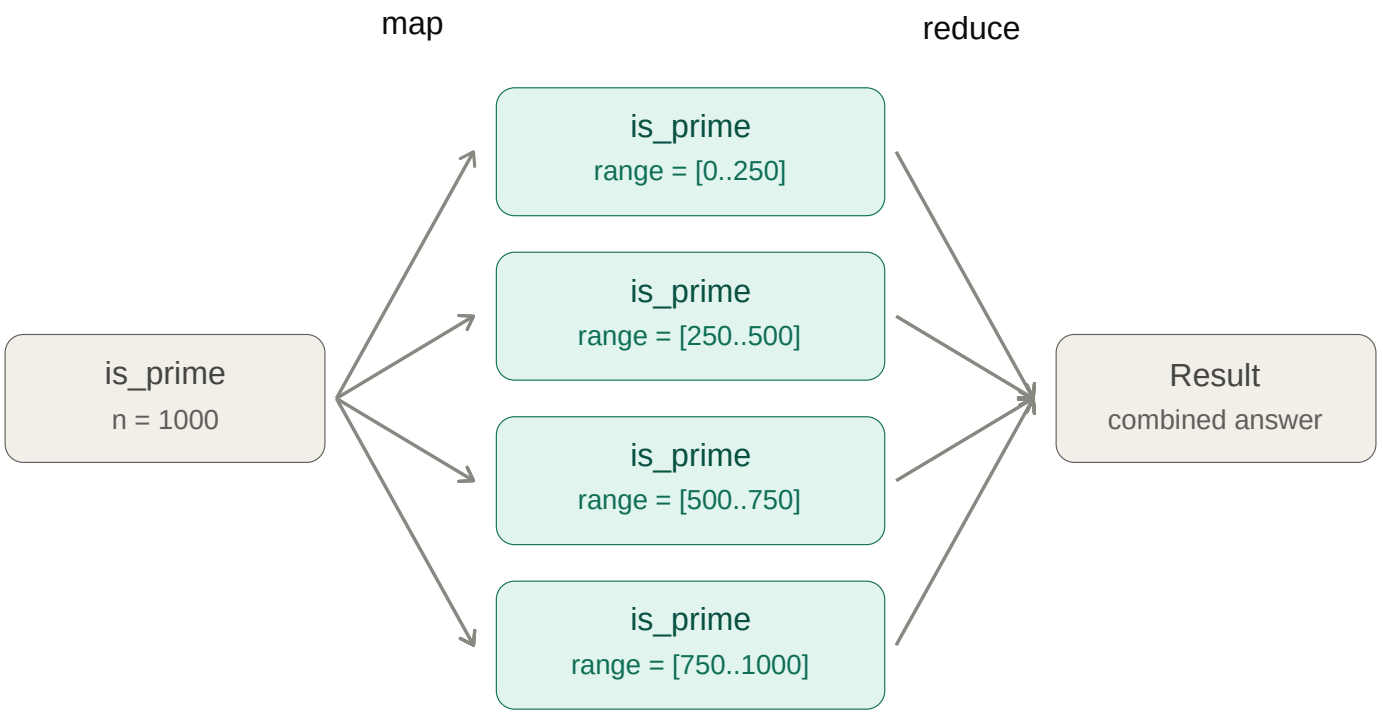


GPU
1,000+ cores

Supercomputer
150,000+ cores

Parallel and distributed computing

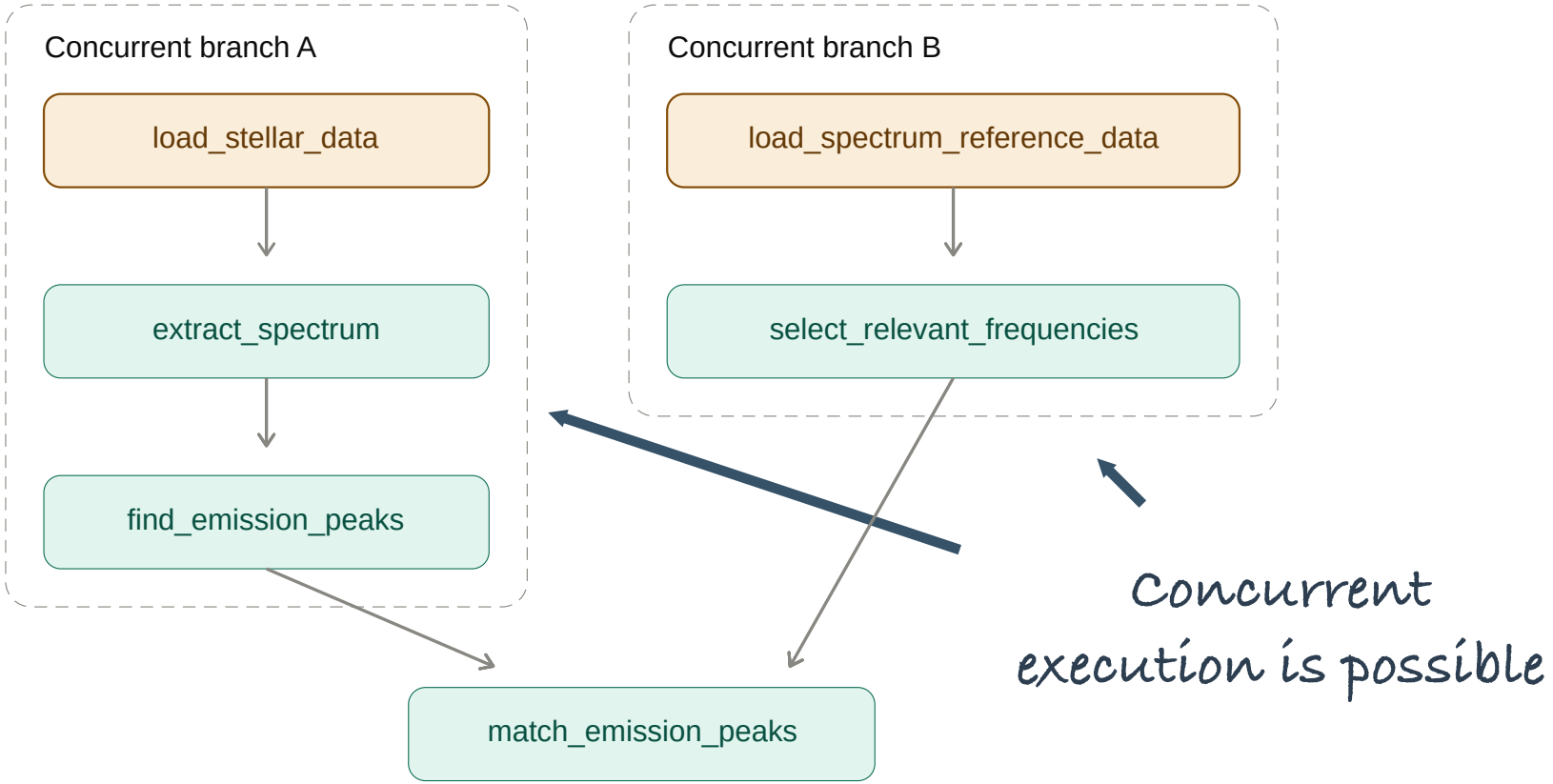
Parallelizing by **splitting the work**



Parallel and distributed computing

```
def analyze_stellar_spectrum(stellar_data_path: str, reference_spectrum: str):
    stellar_data   = load_stellar_data(stellar_data_path)
    spectrum       = extract_spectrum(stellar_data)
    emission_peaks = find_emission_peaks(spectrum)

    reference          = load_spectrum_reference_data(reference_name)
    filtered_reference = select_relevant_frequencies(reference)

    matches = match_emission_peaks(emission_peaks, filtered_reference)

    return matches
```

Parallel and distributed computing



Parallel and distributed computing

Challenges

- **How to split** the task
- Sub-tasks **must be independent** (*functional purity*)
- **Orchestration / synchronization** overhead
- **Computation efficiency** vs **synchronization efficiency**
  - Too low parallelism => low CPU usage
  - Too much parallelism => low performances

Parfun

Parallelize Python functions by **splitting the input data**

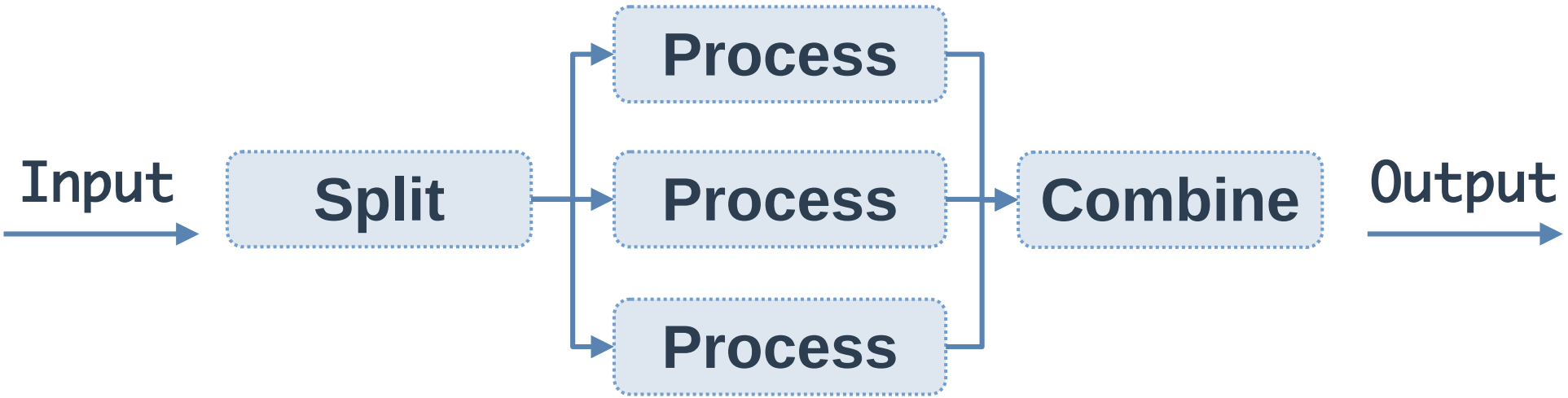
https://github.com/finos/opengris-parfun

Parfun – Count words in text

```
from collections import Counter

def count_words(text: List[str]) -> Counter:
    counter = Counter()
    for line in text:
        for word in line.split():
            counter[word] += 1

    return counter
```

```
>>> count_words(open("small_text.txt").readlines())
Counter({'the': 117,
         'and': 106,
         'of': 90,
         'to': 83,
         'in': 42,
         'right': 33,
        ...})
```

Parfun – Map reduce



Parfun – @parallel

```
import parfun as pf

@pf.parallel(
    # Parallelize by splitting the file content
    split=pf.per_argument(
        lines=pf.py_list.by_chunk

    ),
    # Sum the result counters
    combine_with=sum,
)
def count_words(text: List[str]) -> Counter:
    ...
```

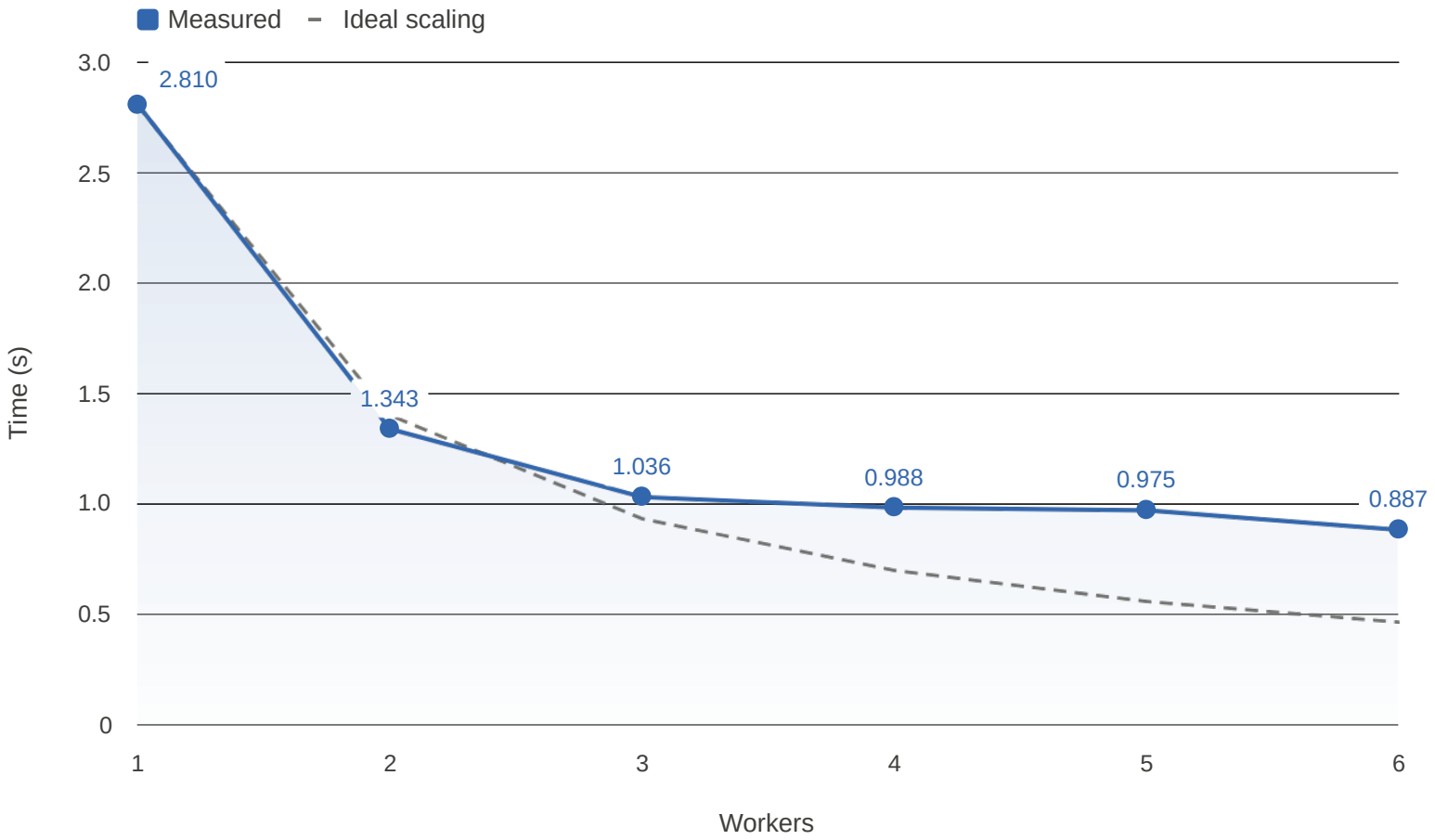
```
>>> count_words(open("very_large_file.txt").readlines())
Counter({'the': 11700,
         ...
```

Parfun – @parallel

```
import parfun as pf

@pf.parallel(
    # Parallelize by splitting the file content
    split=pf.per_argument(
        lines=pf.py_list.by_chunk

    ),
    # Sum the result counters
    combine_with=sum,
)
def count_words(text: List[str]) -> Counter:
    ...
```

```
>>> count_words(open("very_large_file.txt").readlines())
Counter({'the': 11700,
         ...
```

Parfun – Performances



Parfun – Find the optimal batch size

How to find the **optimal sub-task size**?

- Text too **small: overheads will large**
  - Communication, I/O, synchronization ...
- Text too **large: parallelism will be low**

Parfun – Find the optimal batch size

Use **Machine-Learning**!

```
count_words()
   total CPU execution time: 0:00:00.174216.
   compute time: 0:00:00.165855 (95.20%)
      min.: 0:00:00.017239
      max.: 0:00:00.020540
      avg.: 0:00:00.018428
   total parallel overhead: 0:00:00.008361 (4.80%)
      total partitioning: 0:00:00.006238 (3.58%)
      average partitioning: 0:00:00.000693
      total combining: 0:00:00.002123 (1.22%)
   maximum speedup (theoretical): 8.48x
   total partition count: 9
      estimator state: running
        estimated partition size: 1638
```

Pargraph

A declarative **distributed graph engine**

https://github.com/finos/opengris-pargraph/

Pargraph – Declarative graphs

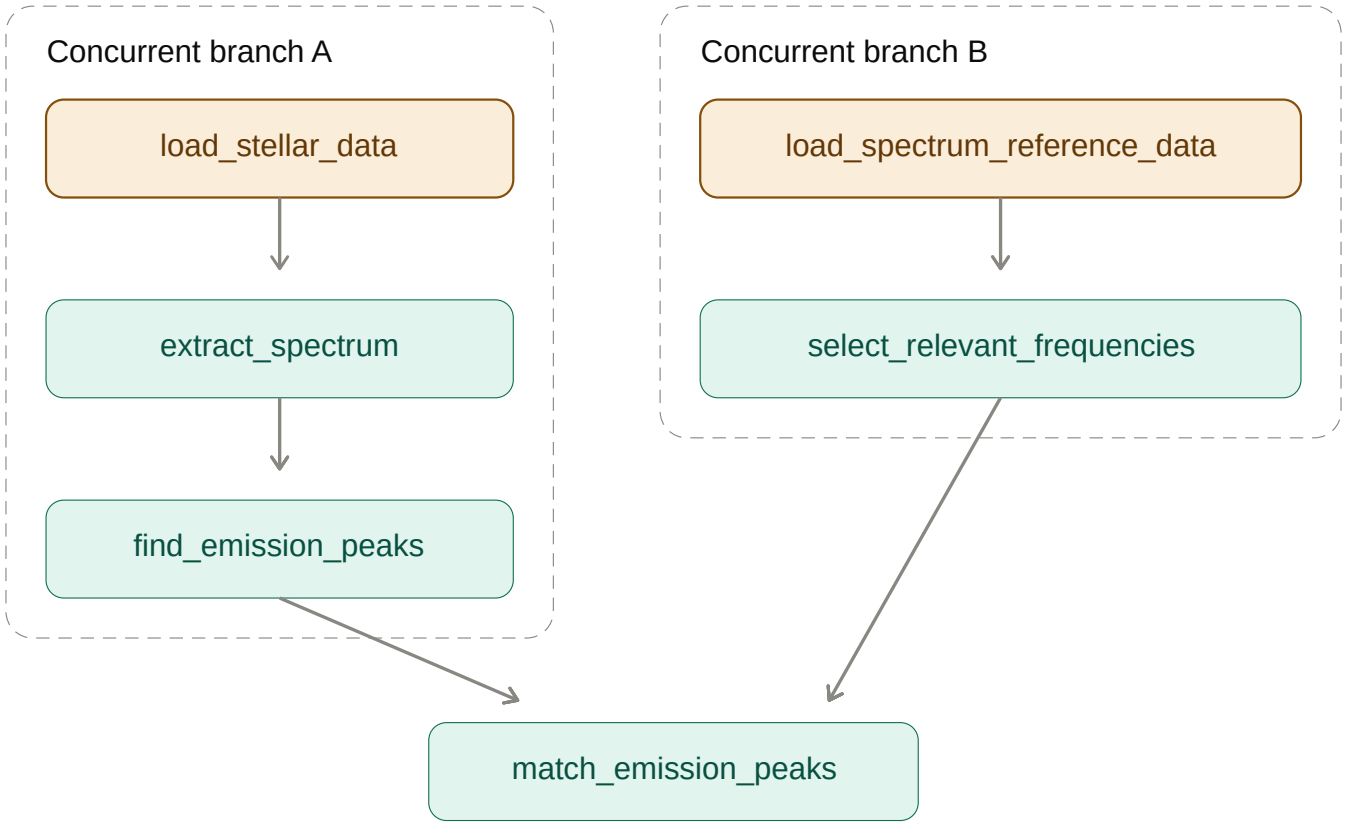
```
def analyze_stellar_spectrum(stellar_data_path: str, reference_spectrum: str):
    stellar_data   = load_stellar_data(stellar_data_path)
    spectrum       = extract_spectrum(stellar_data)
    emission_peaks = find_emission_peaks(spectrum)

    reference          = load_spectrum_reference_data(reference_name)
    filtered_reference = select_relevant_frequencies(reference)

    matches = match_emission_peaks(emission_peaks, filtered_reference)

    return matches
```

Pargraph – Declarative graphs

```
def analyze_stellar_spectrum(stellar_data_path: str, reference_spectrum: str):
    stellar_data   = load_stellar_data(stellar_data_path)
    spectrum       = extract_spectrum(stellar_data)
    emission_peaks = find_emission_peaks(spectrum)

    reference          = load_spectrum_reference_data(reference_name)
    filtered_reference = select_relevant_frequencies(reference)

    matches = match_emission_peaks(emission_peaks, filtered_reference)

    return matches
```

Can run concurrently !

Pargraph – Declarative graphs



Pargraph – Declarative graphs

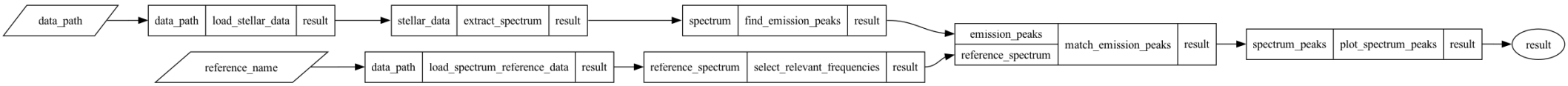
```
from pargraph import delayed, graph

@delayed
def load_stellar_data(data_path: str) -> np.array:
    ...

@delayed
def extract_spectrum(stellar_data: np.array) -> np.array:
    ...

@delayed
def find_emission_peaks(spectrum: np.array) -> np.array:
    ...

...

@graph
def analyze_stellar_spectrum(stellar_data_path: str, reference_spetrum: str):
    ...
```

Pargraph – Declarative graphs

```python
from pargraph import delayed, graph

@delayed
def load_stellar_data(data_path: str) -> np.array:
    ...

@delayed
def extract_spectrum(stellar_data: np.array) -> np.array:
    ...

@delayed
def find_emission_peaks(spectrum: np.array) -> np.array:
    ...

...

@graph
def analyze_stellar_spectrum(stellar_data_path: str, reference_spectrum: str):
    ...
```

Pargraph – Declarative graphs

```
>>> analyze_stellar_spectrum.to_graph().to_dot().write_png("graph.png")
```



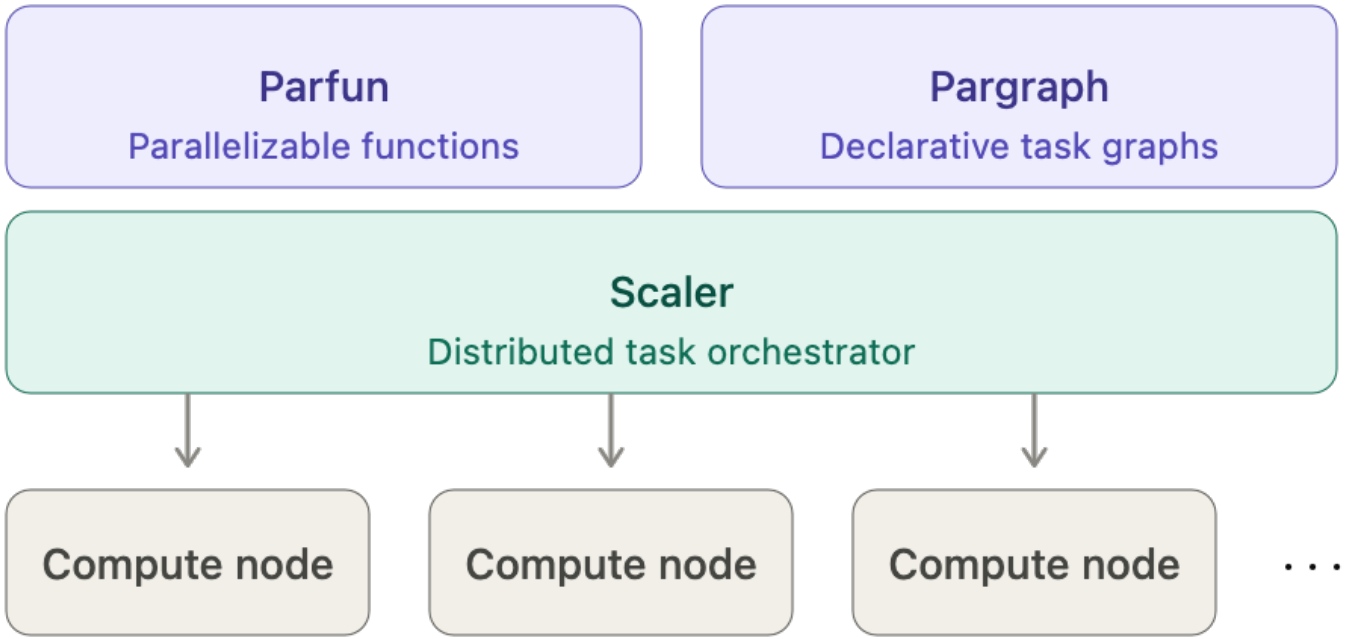

```
>>> graph_engine.get(analyze_stellar_spectrum.to_graph(data_path=..., ...))
```

Scaler

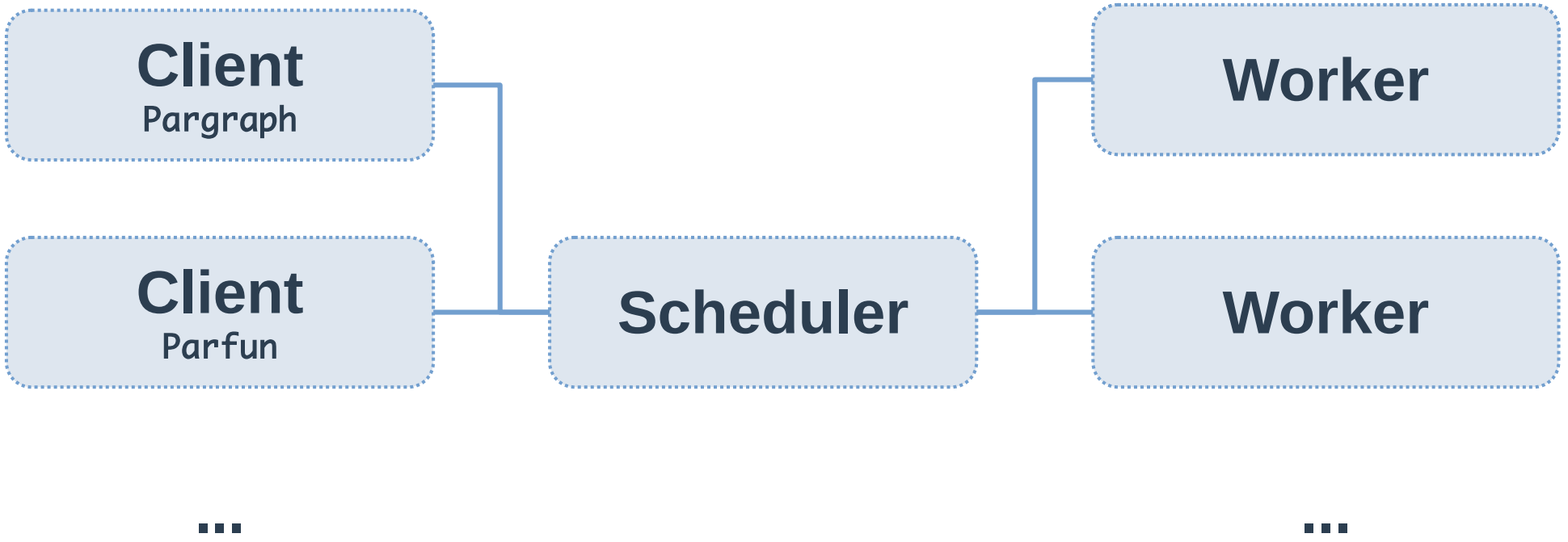
A **distributed task scheduler**

https://github.com/finos/opengris-scaler/

Scaler - Architecture



Scaler - Architecture



Scaler – Setup

Installation

```
$ pip install opengris-scaler
```

Scheduler

```
$ scaler scaler_config.toml
```

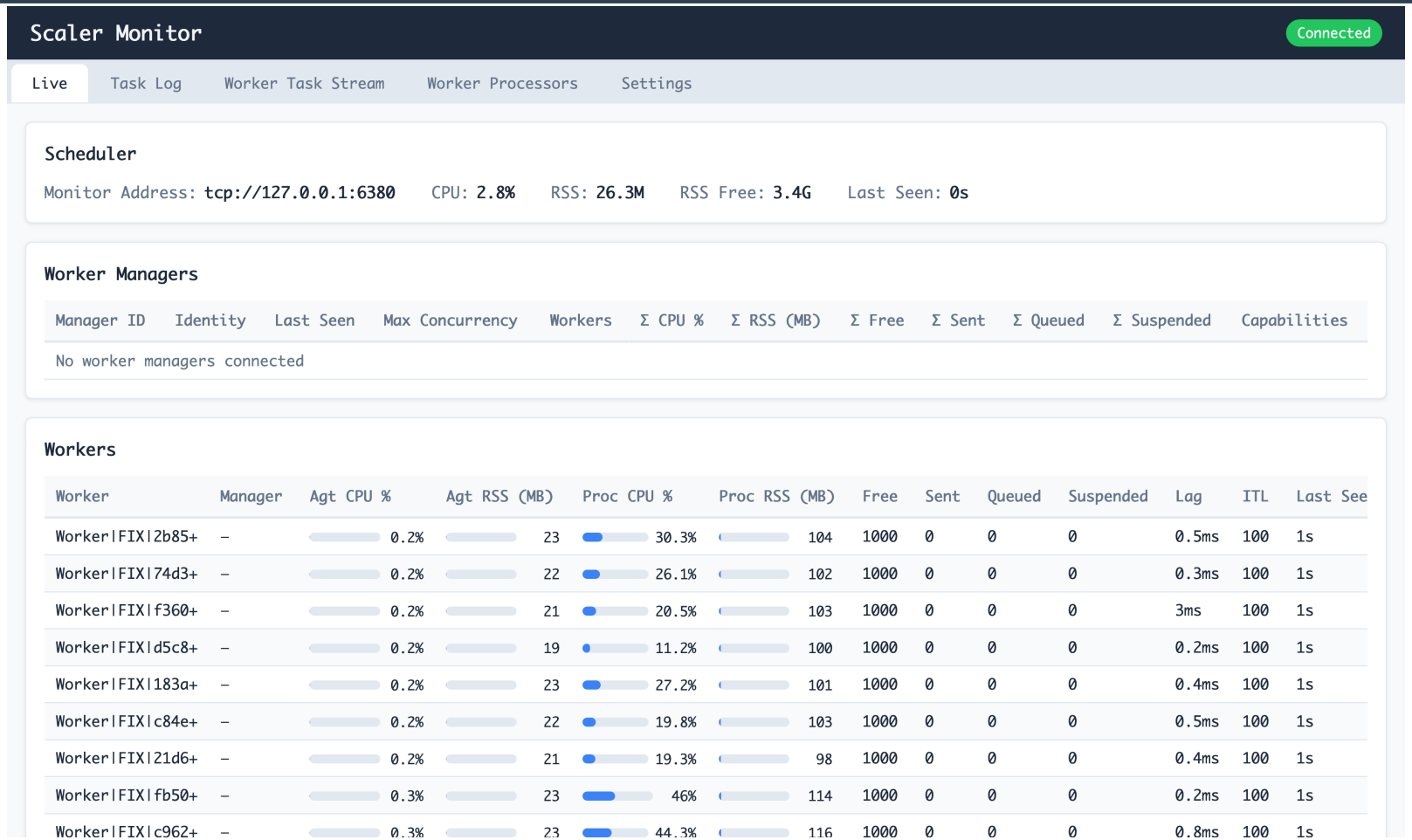
Worker nodes

```
$ scaler_worker_manager baremetal_native "tcp://scheduler_address"
```

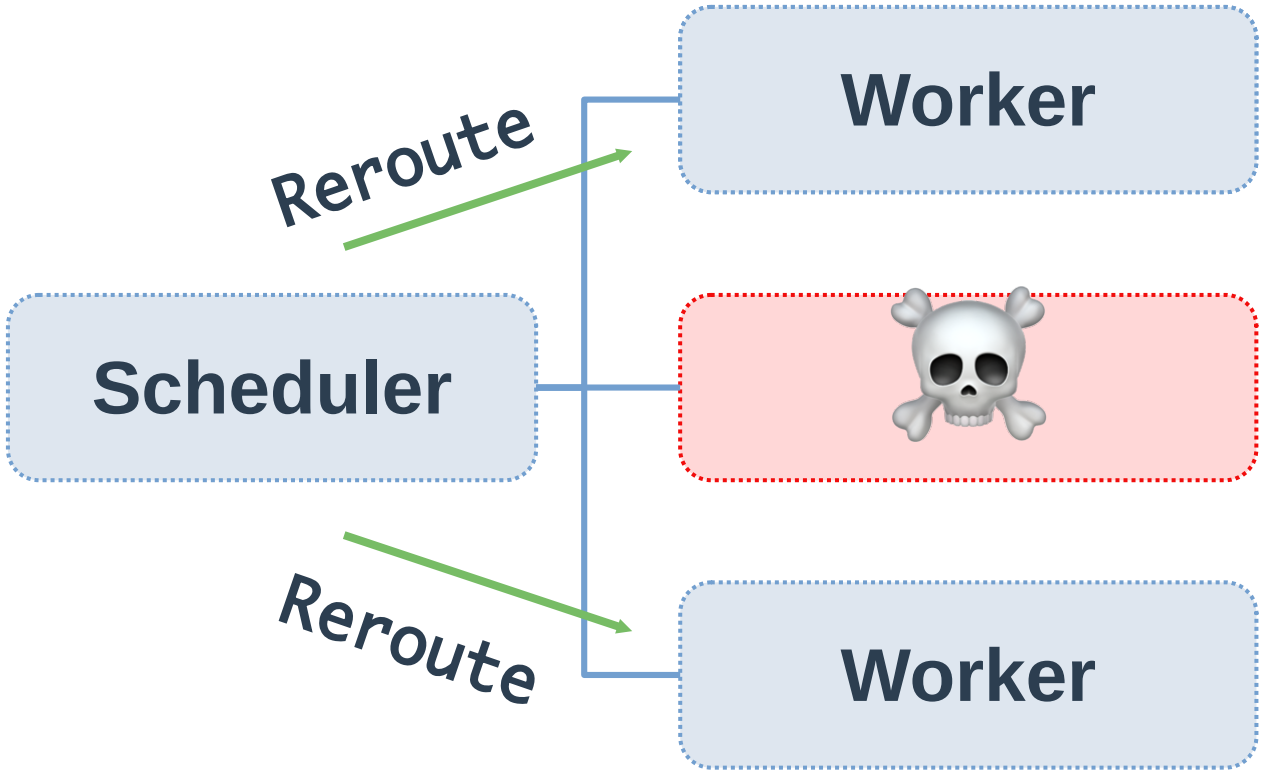
Scaler – Python client

```
# Parfun tasks
with pf.set_parallel_backend_context("scaler_remote", scheduler_address):
    counts = count_words(my_very_long_text)


# Pargraph tasks
from pargraph import GraphEngine
from scaler import Client

graph_engine = GraphEngine(Client(scheduler_address))

graph = analyze_stellar_spectrum.to_graph()
graph_nodes, graph_keys = graph.to_dict(stellar_data_path="stellar_data.npz",
spectrum_reference="spectrum_ref.npz")

graph_engine.get(graph_nodes, graph_keys)
```

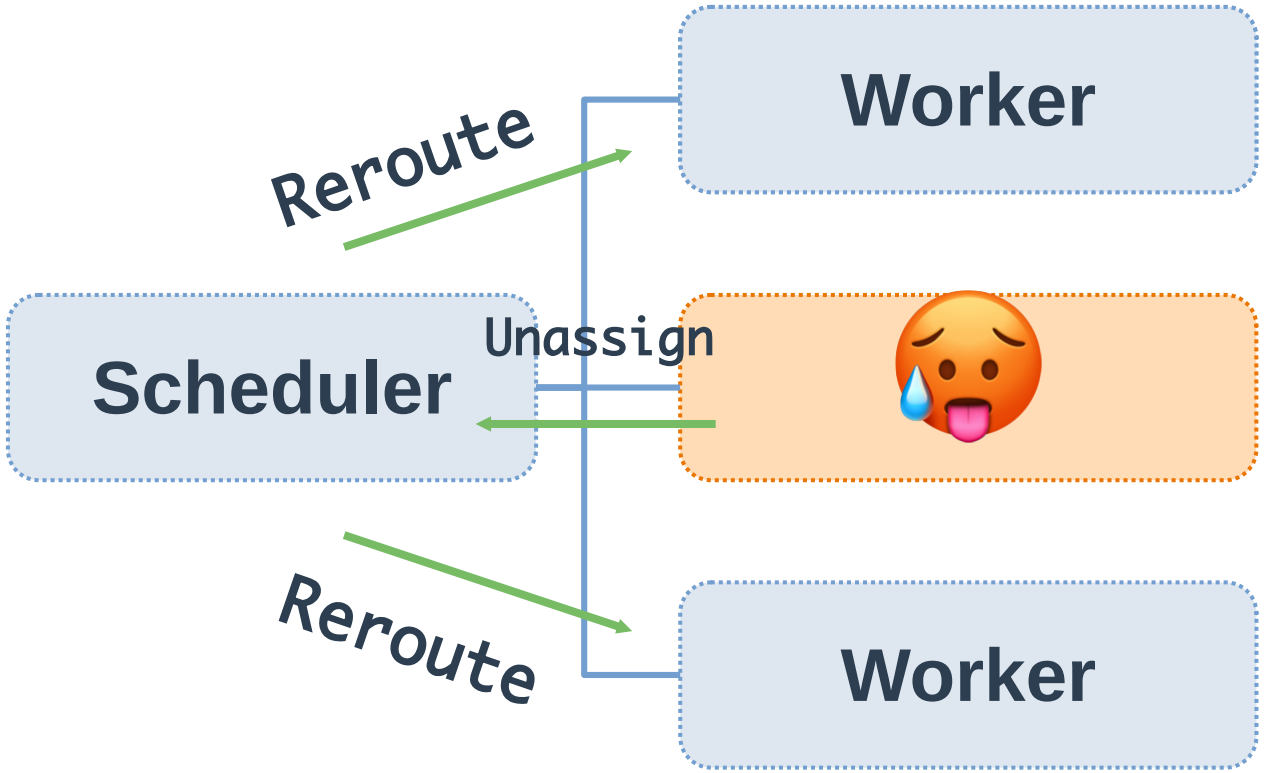
Scaler – Web monitor



Scaler – Failure recovery



Scaler – Dynamic load balancing



Scaler – Warning

- ⚠️ Experimental -- under development
- ⌛ Stable versions of Scaler, Parfun and Pargraph expected soon!

Astrophysics experiment

Thank you !

Q & A

Raphael J.
https://raphaelj.be